

Parallel Machine Learning Algorithms



How modern AI trains faster through parallelism

Megi Sollaku — Foundations & Motivation

Arbiana Muka — Key Algorithms & Techniques

Nikol Isaku — Real-World Systems & Results

What We'll Cover Today

01

Foundations

Why ML needs parallelism,
types of parallelism, neural
network operations

02

Algorithms

Parallel gradient descent,
parameter servers,
AllReduce, sync vs. async

03

Real-World Systems

Google, Meta, Amdahl's Law
applied to ML, challenges &
trade-offs

04

Mini-Project Demo

Sequential vs. parallel
training comparison in
Python

DID YOU KNOW?

Fun Facts About Parallel ML



\$100M+ to train GPT-4

Training GPT-4 is estimated to have cost over \$100 million in compute alone — more than most Hollywood films.



4 quadrillion ops/sec

An NVIDIA H100 GPU's speed — yet still not enough to train large models alone.



Henry Ford knew it first

Data parallelism is essentially the same idea as an assembly line: Henry Ford figured out in 1913 what ML engineers rediscovered in the 2010s



CO₂ cost of one training run

A single training run for a large language model can produce as much CO₂ as five cars over their entire lifetimes.



Backprop is from 1986

The concept behind backpropagation was discovered in 1986 — but models were too small and computers too slow for parallelism to matter. It took 25 years and GPUs designed for video games to make it relevant



The straggler problem = group projects

The "straggler problem" in parallel ML has a direct equivalent in real life: a group project where everyone waits on the one slow member.

Why Is Machine Learning Slow?



Massive Datasets

Modern ML trains on billions of examples. ImageNet alone has 14M images. GPT-3 used 45TB of text.



Huge Models

GPT-4 has ~1 trillion parameters. Each requires storage, computation, and gradient updates.



Iterative Training

Gradient descent requires thousands of passes (epochs) over the full dataset to converge.



Expensive Math

Each forward/backward pass involves billions of floating-point multiplications in matrix form.

Training Time Without Parallelism

~355

days

GPT-3 on 1 GPU

~29

days

ResNet-50 on 1 GPU

~4

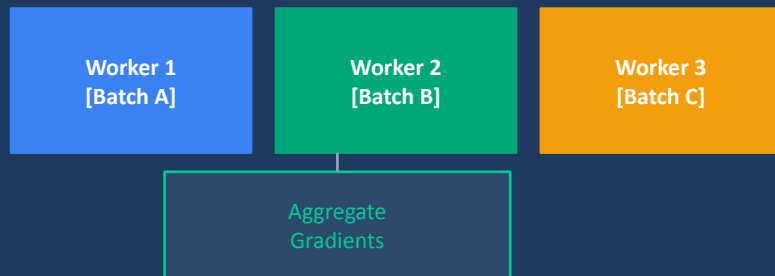
days

BERT-base on 1 GPU

Types of Parallelism in ML

DATA PARALLELISM

Each worker gets a different subset of the training data



- ✓ Most common approach
- ✓ Same model, split data
- ✓ Used by PyTorch DataParallel

MODEL PARALLELISM

The model itself is split across multiple devices



Same data flows through each device sequentially

- ✓ For models too big for one GPU
- ✓ Different layers on different machines
- ✓ Used in LLM training (GPT-4)

Neural Networks Map Naturally to Parallelism

Key Operations



Matrix Multiplication

Each output neuron computed independently — embarrassingly parallel



Batch Processing

Each sample in a batch is independent — can process all at once



Backpropagation

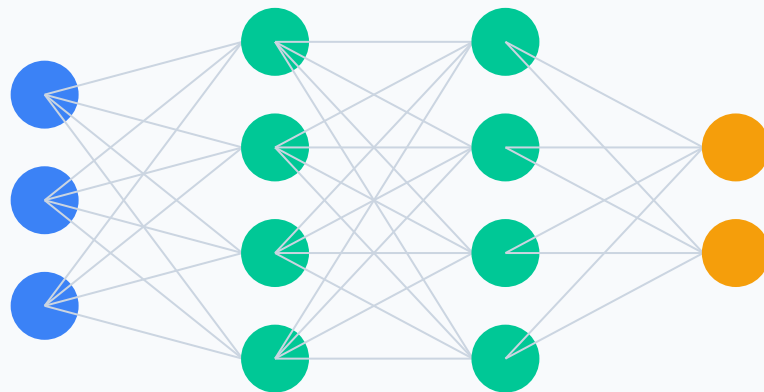
Gradients per layer computed independently after forward pass



Weight Update

Each parameter updated using its own gradient — parallelizable

A Simple Neural Network



Input

Hidden 1

Hidden 2

Output

Key Insight

Every neuron in a layer can be computed simultaneously — this is the heart of ML parallelism.

Parallel Gradient Descent

What is Gradient Descent?

Gradient descent iteratively updates model weights by moving in the direction that reduces the loss function. In standard SGD, this is done one sample at a time — very slow for large datasets.

Sequential**Standard SGD**

1 sample per step. Very slow but low memory.

Baseline**Mini-batch SGD**

Small batches (32–256). Balance of speed & stability.

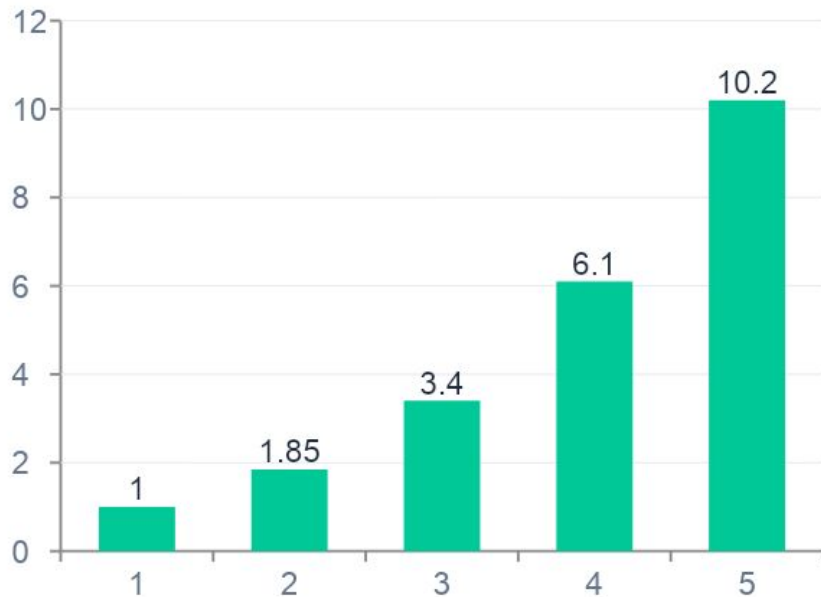
Parallel ✓**Parallel Mini-batch**

Multiple workers process different batches in parallel, gradients averaged.

Async ✓**Asynchronous SGD**

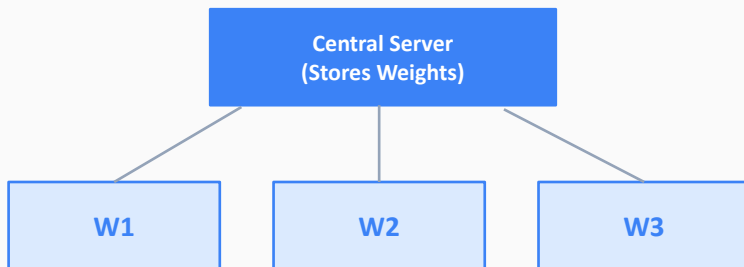
Workers update model without waiting for others. Fastest, but may have stale gradients.

Typical Speedup with Parallel SGD



How Workers Share Gradients

Parameter Server

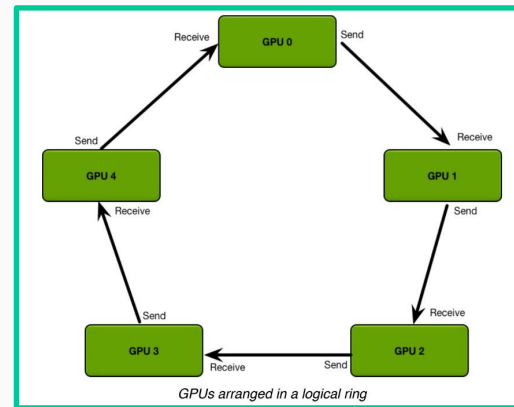


How it works:

1. Workers pull current weights from server
2. Each worker trains on its mini-batch
3. Workers push gradients back to server
4. Server aggregates & updates global model

⚠ *Server can bottleneck*

Ring AllReduce



How it works:

1. Each GPU has full model copy + one data shard
2. GPUs pass gradients around in a ring
3. Each GPU accumulates all gradients
4. All GPUs update weights identically

✓ *No central bottleneck*

Synchronous vs. Asynchronous Training

Aspect	Synchronous	Asynchronous
Update rule	All workers finish before update	Workers update independently
Consistency	Always consistent (exact gradients)	May use stale gradients
Speed	Slower — waits for slowest worker	Faster — no waiting
Convergence	Stable, predictable	Can diverge
Fault tolerance	One failed worker stalls all	System keeps running
Use case	Most modern deep learning (DDP)	Large-scale async RL, Hogwild!

How Companies Scale ML Training

Google

64 TPUs → 76 min BERT

TPU Pods — Tensor Processing Units

Trains BERT in 76 minutes using 64 TPUs. Custom hardware designed for matrix math. Used for Search, Translate, and Gemini.

Meta

16,000 GPUs / job

PyTorch FSDP + Collective I/O

Fully Sharded Data Parallel splits model state across GPUs. Used to train LLaMA-3. Up to 16,000 GPUs in one training job.

OpenAI

1,024 A100 GPUs

Megatron + DeepSpeed + ZeRO

Pipeline parallelism + tensor parallelism + data parallelism combined. GPT-3 used 1,024 A100 GPUs for ~34 days.

Amazon

Cloud-managed scale

SageMaker Distributed Training

Managed cloud service abstracting parallelism strategy. Supports data + model + hybrid parallelism across EC2 instances.

Amdahl's Law & Why Speedup Has Limits

Amdahl's Law:

$$\text{Speedup} = 1 / (S + (1-S)/N)$$

S = fraction of task that is sequential

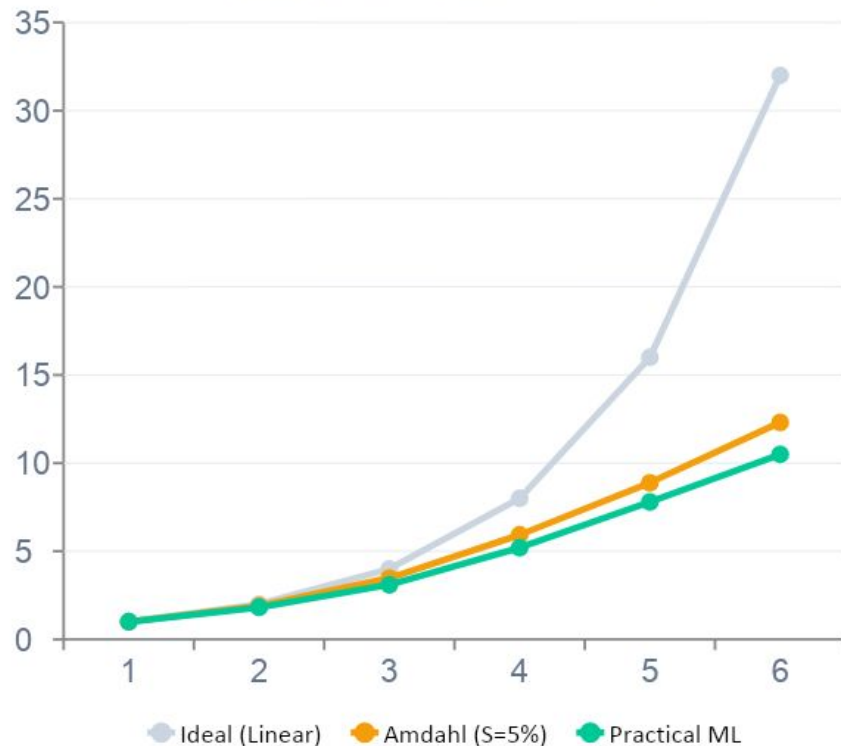
N = number of parallel workers

Even if $S = 5\%$, max speedup $\approx 20\times$, even with ∞ workers

Sequential Bottlenecks in Real ML Systems:

- Data loading & preprocessing (CPU-bound)
- Gradient communication between workers
- Weight update step (synchronized barrier)
- Checkpointing and evaluation passes

Speedup vs Number of Workers



Challenges in Parallel ML Training



Communication Overhead

Sending gradients between hundreds of GPUs is expensive. With 1,000 workers and a 1B-parameter model, each sync requires ~4GB of data transfer.

Solution: Gradient compression, quantization, and bucketed AllReduce



Straggler Problem

In synchronous training, the whole cluster waits for the slowest worker. One faulty or overloaded machine slows everyone down significantly.

Solution: Backup workers, timeout-based exclusion, or switch to async training



Memory Constraints

Large models don't fit in a single GPU's memory. GPT-3 (175B params) needs ~700GB RAM — far more than any single GPU provides.

Solution: Model sharding (ZeRO), gradient checkpointing, CPU offloading



Consistency & Convergence

Asynchronous updates with stale gradients can cause the model to diverge or learn incorrectly, especially with many workers.

Solution: Bounded staleness, synchronization checkpoints, learning rate warmup

Sequential vs. Parallel Training Demo

parallel_train.py

```
from multiprocessing import Pool, cpu_count
import
time, numpy as np

def train_worker(batch):
    # Simulate gradient computation

    time.sleep(0.1) # 100ms per batch

    return np.random.rand(100) # fake gradient

batches = [i for i in range(32)] # 32 batches

# Sequential
t0 = time.time()
[train_worker(b) for b in batches]
seq_time = time.time() - t0

# Parallel
t0 = time.time()
with Pool(cpu_count()) as p:
    p.map(train_worker, batches)
par_time = time.time() - t0
```



Expected Results

Sequential

~3.2s

32 batches × 100ms

Parallel (4 cores)

~0.9s

4 workers sharing work

Speedup

~3.6x

Less than ideal (overhead)

Summary

1

ML is computationally expensive

Training large models sequentially takes months. Parallelism is not optional — it's essential.

2

Two core strategies

Data parallelism splits the data; model parallelism splits the model. Modern systems combine both.

3

AllReduce is the standard

Ring AllReduce (used in PyTorch DDP and NCCL) gives the best balance of speed and no central bottleneck.

4

Speedup has limits (Amdahl's Law)

Sequential portions, communication, and straggler delays set hard limits on parallel efficiency.

5

Industry pushes the frontier

Google, Meta, OpenAI use thousands of specialized accelerators with custom communication libraries.

Thank you! Questions?